

4. 바이트코딩 개요



1) 바이트 코딩



바이브 코딩(Vibe Coding)이란?

- AI 코딩 에이전트와 대화를 주고 받으면서, 바이브(느낌)에 따라 코딩하는 개발 방식을 의미합니다.
 - 2025년 2월, 오픈마 공동 창업자 안드레이 카파시(Andrej Karpathy)가 제안
 - 핵심 : "코딩 경험이 없어도 아이디어만 있으면 프로그램을 만들 수 있다"

기존 방식

- HTML, CSS, JS 문법 학습
- 직접 코드 작성
- 디버깅 및 수정

바이브 코딩 방식

- "To Do List 웹 앱 만들어줘"
- AI가 코드 작성
- 수정 사항 요청

안드레 카파시(Andrej Karpathy)의 바이브코딩 관련 트윗



 Andrej Karpathy ✓
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

새로운 종류의 코딩을 나는 "바이브 코딩(vibe coding)" 이라고 부른다. 이걸 그냥 바이브(느낌)에 완전히 몸을 맡기고, 지수적(exponential) 발전을 즐기면서, 코드가 있다는 사실조차 잊어버리는 방식이다. 이렇게 가능한 이유는 LLM들(예: Cursor Composer + Sonnet)이 너무 좋아졌기 때문이다. 게다가 나는 SuperWhisper를 통해 Composer에게 그냥 말만 하면 되니까 키보드에도 거의 손을 안 댄다.

나는 너무 귀찮아서 "사이드바의 패딩을 절반으로 줄여줘" 같은 가장 바보 같은 요구사항도 그냥 말로 한다. 무조건 "모두 수락(Accept All)" 버튼을 누르고, 코드 차이점(diff) 같은 건 읽지도 않는다. 에러 메시지가 뜨면 아무런 코멘트 없이 메시지를 복사해서 붙여넣기만 하는데, 보통 그렇게 하면 해결된다. 코드는 이미 내가 평소에 이해하는 수준을 넘어 성장해 버렸고, 제대로 이해하려면 정말 시간을 들여서 읽어야 할 지경이다.

가끔 LLM이 버그를 못 잡으면, 그냥 그 주변을 우회하거나, 랜덤하게 뭔가 변경해 달라고 요청해서 버그가 사라질 때까지 반복하기도 한다. 주말에 재미삼아 만드는 프로젝트 정도라면 이 방식도 나쁘지 않다. 오히려 꽤나 재미있다. 뭔가 프로젝트나 웹앱을 만들고는 있지만, 이것 정말 코딩이라고 할 수는 없다-그냥 보고, 말하고, 실행하고, 붙여넣기만 하면, 대체로 다 돌아가기 때문이다.

바이트 코딩으로 인한 소프트웨어 엔지니어의 역할 변화

■ 제품 중심 사고:

- "소프트웨어 엔지니어의 역할이 제품 엔지니어로 전환되고 있습니다. AI 코딩 도구가 모든 사람을 10배 더 뛰어난 엔지니어로 만들면서 인간의 감각이 그 어느 때보다 중요해졌습니다." - Outlet 창업자

■ 코드에 대한 애착 감소:

- 개발자들은 자신이 작성한 코드에 집착하지 않고 코드를 쉽게 버리고 다시 작성하면서 제품 개선에 더 집중

■ 기하급수적 속도 향상:

- "6개월 전에서 1개월 전까지는 10배 속도 향상, 1개월 전에서 지금까지는 100배 속도 향상을 경험했습니다. 더 이상 엔지니어가 아니라 제품 담당자입니다." - Train Loop 창업자

바이브코딩 vs 전통 개발

항목	전통 개발	바이브코딩
주 작업	코드 직접 작성	자연어 프롬프트 작성
필요 지식	프로그래밍 언어 숙달	목표를 명확히 설명하는 능력
속도	기능 하나에 수 시간~수일	분~시간 단위로 프로토타입 완성
오류 처리	직접 디버깅	에러 메시지를 AI에 붙여넣기
코드 이해도	모든 코드를 직접 이해	결과 동작에 집중, 코드 이해는 선택

바이브코딩의 작동 원리

■ 자연어 이해 (Natural Language Understanding)

- LLM은 “깔끔하게”, “사용자 친화적으로”, “부드럽게” 같은 추상적인 표현을 이해하고, 이를 CSS 속성 (예: padding, box-shadow), 애니메이션, 컴포넌트 구조 등 구체적인 코드로 변환할 수 있습니다.

■ 맥락 인식 (Context Awareness)

- Cursor와 같은 AI 코딩 도구는 현재 열려 있는 파일이나 프로젝트 전체의 코드를 맥락으로 파악합니다. 따라서 “여기 있는 버튼 스타일을 다른 페이지에도 적용해줘”와 같은 요청을 이해하고 일관성 있는 코드를 생성합니다.

■ 반복적 개선 (Iterative Refinement)

- 바이브 코딩은 한 번의 명령으로 끝나지 않습니다.
- LLM이 생성한 결과물을 보고, “좀 더 부드러운 그림자 효과를 줘” 또는 “애니메이션 속도를 약간만 빠르게 해줘”와 같이 대화를 통해 점진적으로 결과물을 완성해 나가는 과정입니다.

바이브코딩의 핵심 원칙(1)

- 의도를 먼저, 구현은 AI에게
 - 무엇을 만들고 싶은지 명확하게 정의하는 것이 출발점입니다.
 - “로그인 버튼을 만들어줘”가 아니라 “이메일과 비밀번호를 입력받아 JWT 토큰을 발급하고 로컬 스토리지에 저장하는 React 로그인 폼을 만들어줘”처럼 구체적으로 설명할수록 결과물의 품질이 높아집니다.
- 빠른 실행 → 피드백 → 반복
 - 코드를 받으면 즉시 실행해 결과를 확인합니다. 원하는 결과와 다르면 AI에게 “이 부분을 이렇게 바꿔줘”라고 요청합니다.
 - 이 루프를 빠르게 반복하는 것이 바이브코딩의 리듬입니다.
- 에러는 복사해서 AI에 붙여넣기
 - 오류가 발생하면 에러 메시지를 직접 분석하지 않고 AI에게 그대로 전달합니다.
 - “이런 에러가 나왔어, 고쳐줘”라고 하면 AI가 원인을 파악하고 수정된 코드를 제공합니다.

바이트코딩의 핵심 원칙(2)

■ 큰 기능은 작게 쪼개기

- 한 번에 전체 앱을 만들어달라고 하면 AI가 복잡한 코드를 생성해 오류가 많아집니다.
- 기능을 작은 단위로 나눠 순서대로 요청하면 각 단계에서 동작을 확인하며 진행할 수 있습니다.

■ 컨텍스트 유지

- AI는 대화 맥락을 기억합니다.
- 새로운 기능을 요청할 때 "방금 만든 로그인 기능에 소셜 로그인도 추가해줘"처럼 이전 작업을 참조하면 AI가 기존 코드 구조를 이해한 상태에서 수정해줍니다.

바이트코딩 방법론

- 1) 소프트웨어를 기획한다.
- 2) AI에게 기획을 주입하고 세부 작업 단위를 분해하도록 명령한다.
- 3) 작게 분해된 각 작업 단위를 AI가 순서대로 작업하도록 명령한다.
- 4) 완성된 코드를 실행한다.
- 5) 오류가 생기면 오류 메시지를 복사해서 AI에게 디버깅하도록 명령한다.
- 6) 원하는 수준의 소프트웨어가 완성될때까지 이 과정을 반복한다.

바이트 코딩의 장점

- 개발 속도 급증:

- 개발자들은 AI 도구를 사용해 이전보다 최대 100배 빠르게 코드를 작성할 수 있습니다.

- 진입장벽 감소:

- 수학, 물리학 같은 기술적 배경을 가진 사람들이 전통적인 프로그래밍 교육 없이도 소프트웨어 개발에 참여할 수 있게 되었습니다.

- 제품과 사용자 경험 중심:

- 코드 작성에 시간을 덜 쓰게 되면서 개발자들은 제품과 사용자 경험 개선에 더 집중할 수 있습니다.

바이트 코딩의 한계

■ 디버깅의 어려움:

- 현재 AI 모델들은 코드 생성에는 뛰어나지만 디버깅에는 여전히 약점을 보이고 있어, 복잡한 버그를 찾고 해결하는 데는 인간의 깊은 이해가 필요합니다.

■ 시스템 설계와 확장성 문제:

- 바이트 코딩은 프로토타이핑(0 to 1)까지는 뛰어나지만, 수백만 사용자를 지원하는 규모로 확장(1 to N)하는 데는 한계가 있습니다.

■ 코드 품질 판단능력:

- 좋은 코드와 나쁜 코드를 구별하는 "느낌"은 실무 경험에서 비롯되며, AI가 생성한 코드를 제대로 이해하고 품질을 평가할 수 있는 능력이 여전히 중요합니다.

2) AI 코드 어시스턴트



AI 코드 어시스턴트란?

- AI 코드 어시스턴트는 인공지능과 머신러닝 기술을 활용하여 소프트웨어 개발자와 프로그래머에게 코드 작성, 디버깅, 최적화 등의 과정을 지원하는 도구입니다.
 - 자연어 처리(NLP)와 머신러닝 기술을 활용하여 사용자의 입력을 이해하고, 적절한 코드 조각을 제안하거나 자동으로 생성하는 기능을 제공합니다.
 - 단순한 코드 자동완성을 넘어 컨텍스트를 이해하고, 전체 함수나 컴포넌트를 생성하며, 버그를 감지하고 수정 제안까지 해주는 진정한 '페어 프로그래머' 역할을 합니다.
 - 이러한 도구들은 개발 시간을 크게 단축시키고 코드 품질을 향상시키는 필수 도구로 자리잡았습니다.

코딩 에이전트의 핵심 특징 및 기능

- 자율성: 최소한의 인간 개입으로 목표 설정 및 코드 실행 (작성 에러 수정 완성).
- 맥락 이해: 파일, 클래스, 함수 등 프로젝트의 전체적인 구조와 흐름을 파악.
- 도구 활용: 터미널 명령 실행, 코드 검색, 문서화, 테스트 스크립트 실행 등의 도구를 직접 사용.

주요 사용 예시 (Usage Examples)

- 자동 코드 생성 및 리팩토링: 파이썬 코드를 자바 코드로 변환하거나 코드의 품질 개선.
- 테스트 주도 개발(TDD): 코드를 작성하고 해당 코드의 테스트를 자율적으로 생성 및 통과.
- 버그 수정: 오류 메시지를 분석하여 소스 코드를 자율적으로 수정.
- 코드 리뷰: 코드의 스타일, 보안, 복잡도 등을 리뷰하고 개선 제안.
- 프로젝트 생성: 상위 수준의 요구사항을 입력받아 초기 프로젝트 구조 세팅.

좋은 코딩 에이전트의 기준

- 컨텍스트를 이해하고 여러 파일에 걸쳐 동작하며, 개발자가 실제로 소프트웨어를 만드는 과정을 잘 통합
 - 현재 파일뿐 아니라 프로젝트 전체 구조를 이해함
 - 단순한 제안이 아닌 실제 코드 변경이 가능함
 - 작업 흐름에 방해되지 않음
 - 지루한 반복 작업을 대신 처리해주어 흥미로운 문제에 집중할 수 있게 해줌

Cursor

- Cursor는 VS Code에서 포크된 완전한 코드 에디터로, 개발 경험의 모든 측면에 AI가 깊이 통합되어 새로운 세대의 AI 강화 IDE를 대표합니다.
- 강점:
 - AI 네이티브 에디터 경험
 - 자율적인 다중 파일 변경을 위한 에이전트 모드
 - 탁월한 코드베이스 이해
 - Tab 완성이 자연스럽게 컨텍스트를 인식
- 한계:
 - 현재 IDE에서 전환 필요
 - 에이전트 모드의 학습 곡선



Cursor AI

ChatGPT

- ChatGPT(GPT-5)는 사용 가능한 가장 다재다능하고 유연한 코딩 어시스턴트로 남아 있습니다. 일상적인 개발, 문제 해결, 다양한 일반 작업을 일관된 신뢰성으로 처리합니다.
- 강점:
 - 벤치마크에서 95% 코드 정확도
 - 빠르고 반응성 좋음
 - 처음부터 새 코드 생성에 탁월
 - 모든 작업 유형에 걸쳐 강력한 추론
- 한계:
 - 더 작은 128K 토큰 컨텍스트 윈도우
 - 채팅 인터페이스로 코드 복사-붙여넣기 필요



GitHub Copilot

- GitHub Copilot은 수십억 줄의 코드로 훈련되었으며, VS Code, JetBrains IDE, Neovim에 직접 통합되어 타이핑하는 동안 완성을 제안합니다.
- 장점:
 - 원활한 IDE 통합
 - 자동 완성 및 인라인 제안에 탁월
 - 일반적인 패턴과 프레임워크에 대한 강한 이해
 - 학생 무료, 개인 월 \$10
- 한계:
 - 컨텍스트 윈도우가 현재 파일과 열린 탭으로 제한
 - 복잡한 아키텍처 결정에는 덜 효과적
 - 오래되거나 deprecated된 패턴을 제안할 수 있음



Anthropic Claude Code

- Claude Code 핵심 기능
 - 커맨드라인 인터페이스 기반 통합 워크플로우
 - 최첨단 추론 능력을 보여주는 Claude Sonnet 4 탑재
 - 자율적인 작업 완수
 - 개발 파이프라인과의 연동
- Claude의 최적 활용 예시
 - 자율적인 코딩 지원이 필요한 커맨드라인 기반 고급 개발자



Google Gemini Code Assist

- Gemini는 멀티모달 워크플로우에서 빛나며 Google Workspace와의 깊은 통합을 제공합니다. 100만 토큰 컨텍스트 윈도우는 대규모 코드베이스를 잘 처리합니다.
- 강점:
 - 거대한 컨텍스트 윈도우
 - 강력한 UI/UX 및 비주얼 디자인 지원
 - 멀티미디어 및 크로스모달 프로젝트에 탁월
 - Google 생태계와의 깊은 통합
- 한계:
 - 순수 텍스트 추론에서는 약간 뒤처짐
 - 코드 품질이 덜 일관적
 - Google Cloud 사용자에게 가장 적합



코딩 에이전트 활용 팁

- 자기 워크플로우부터 고민할 것:

- 모든 걸 바꾸도록 강요하는 도구는 피하고 이미 하는 방식과 맞는 도구를 고릅니다.

- 맹신하지 말 것:

- 강력하지만 완벽하진 않은 생성 코드의 정확성과 보안을 꼭 검토하세요.

- 반복 업무에 활용할 것:

- 코딩 에이전트는 지루한 부분을 대신해주어, 더 흥미로운 문제에 집중할 수 있게 해줍니다.

- 항상 주도권 유지:

- 이 도구들은 능력을 증폭시켜주는 파트너이지, 개발자를 대체하도록 만들어진 것이 아니므로 최종판단은 사용자의 몫입니다.

코딩 에이전트의 미래

- 전체 프로젝트 맥락의 이해력 향상
- 더 자율적인 작업 완수 (일일이 지시할 필요 없음)
- 특정 도메인/사용 사례용 전문화 에이전트 등장
- AI와 인간 개발자 간의 협업 강화



THANK YOU!